Gráf-adatbázisok

Vassányi István
Pannon Egyetem, Veszprém

Az alapgondolat

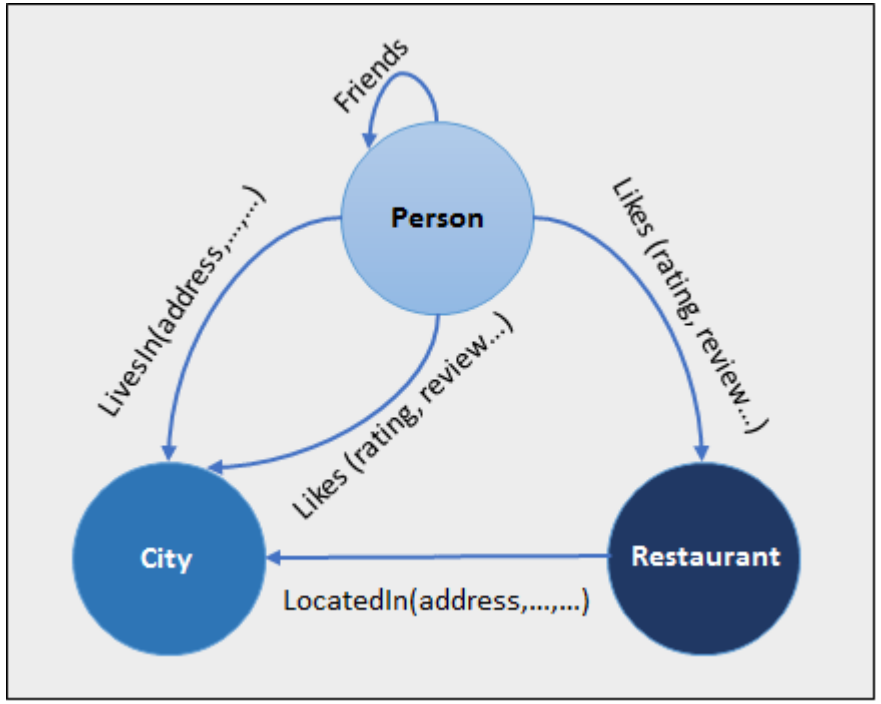
- Az adatbázis-tervezés a domain modellel kezdődik
  - UML osztálydiagram: asszociáció, aggregáció, kompozíció, öröklődés
  - EER diagram: különböző kardinalitású kapcsolatok, specializáció/általánosítás
- Relációs logikai adatmodell:
  - 3 féle kapcsolat: 1:1, 1:N, N:M az összes fenti esetre (egyszerűsítés)
  - 1:1, 1:N -> külső kulcs, N:M -> kapcsoló tábla
- Gráf adatbázis: hálózatos adatmodell
- AZ ADATBÁZISBAN CSAK ÉLEK ÉS CSOMÓPONTOK VANNAK
  - Élek: irányított vagy irányítatlan, attribútumok
  - Csomópontok: azonosítók, attribútumok

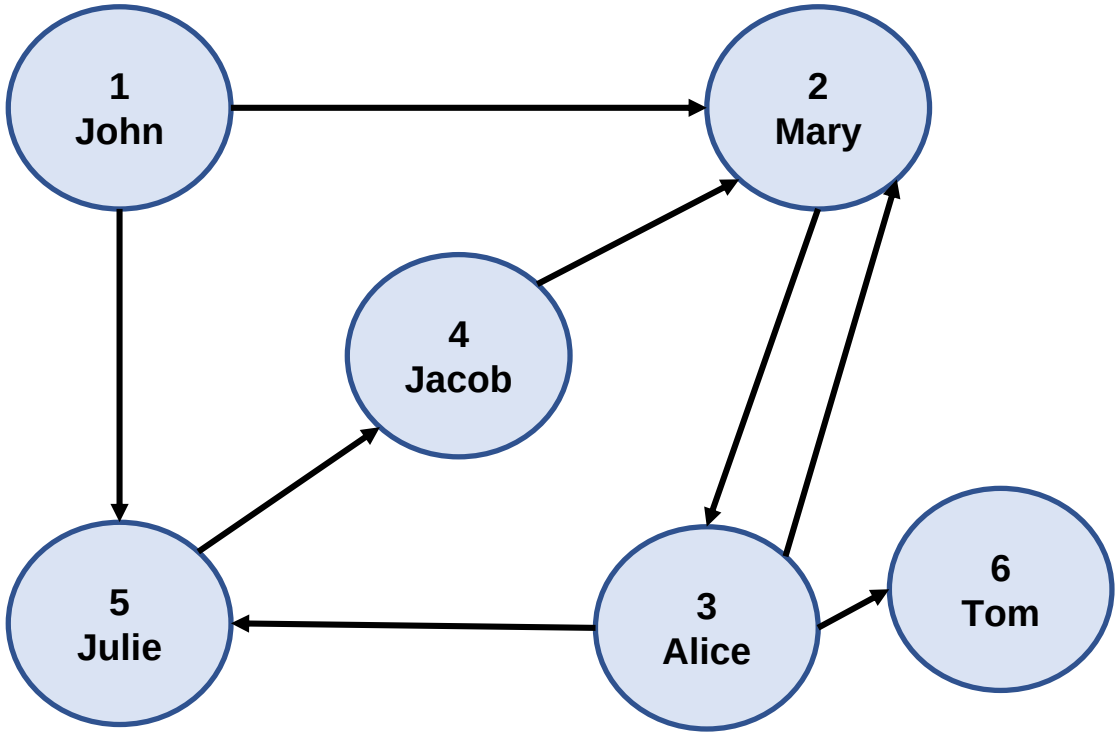
Gráf-adatbázis, mint relációs kiterjesztés

- Táblák létrehozása: as node, as edge
- Automatikusan generált azonosítók
  - $node_id, $edge_id, $from_id, $to_id
- Az élekre típus-kényszerek helyezhetők el
- Nincs irányítatlan él
- Megfeleltethető a relációs sémának
- Lekérdezés keresési mintázattal (match pattern)
  - Intuitív, könnyen érthető, népszerű

DEMO

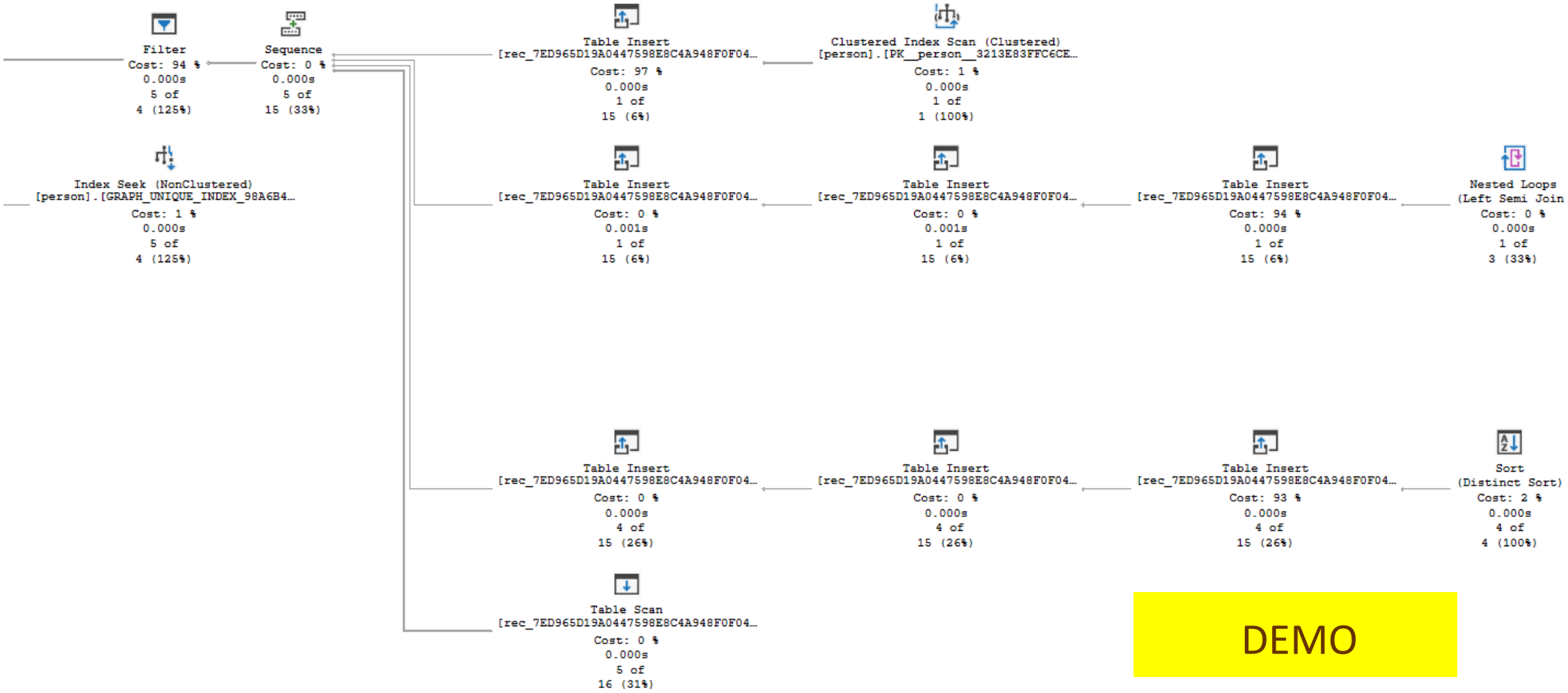
A demo adatbázis





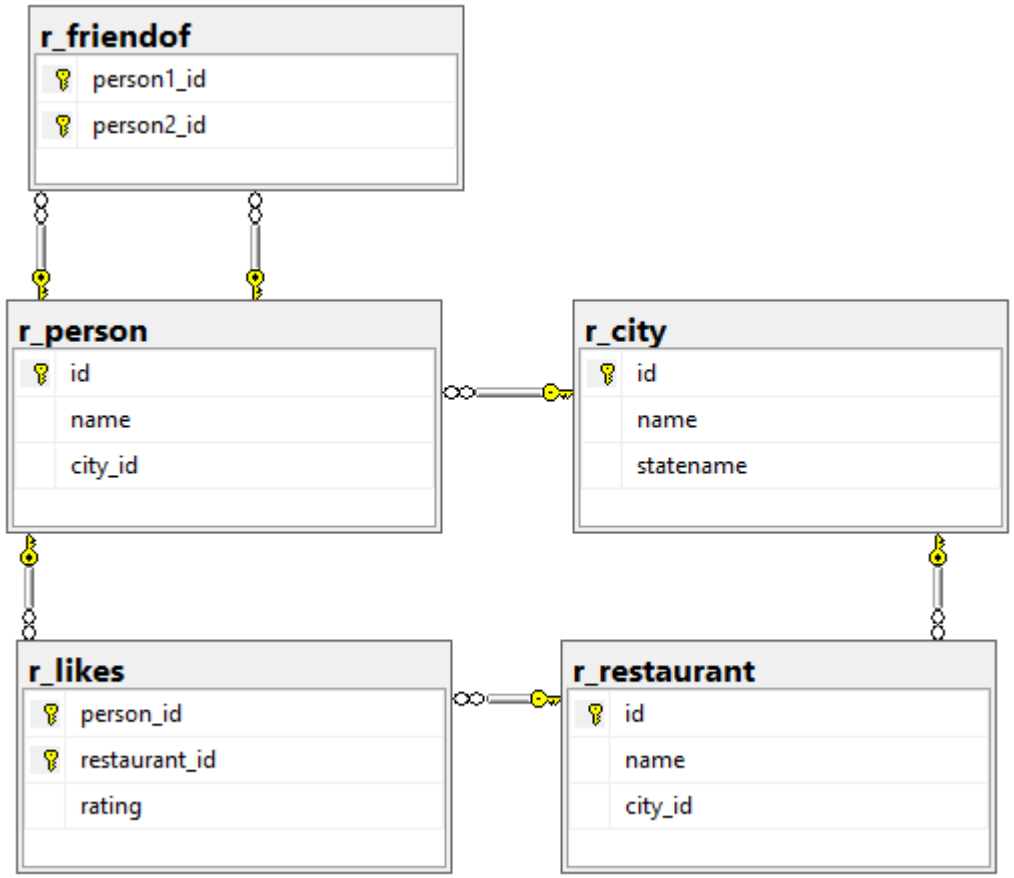
A legrövidebb ösvény végrehajtási terve

Query 1: Query cost (relative to the batch): 100%
SELECT name, paths, depth FROM (SELECT p1.name, p1.name + '->' + string_agg(p2.name, '->') WITHIN GROUP (GRAPH PATH) AS paths, last_value(p2.Name) WITHIN GROUP (GRAPH P…



DEMO

Ugyanez relációs modellben



DEMO

Relációs->gráf átalakítás

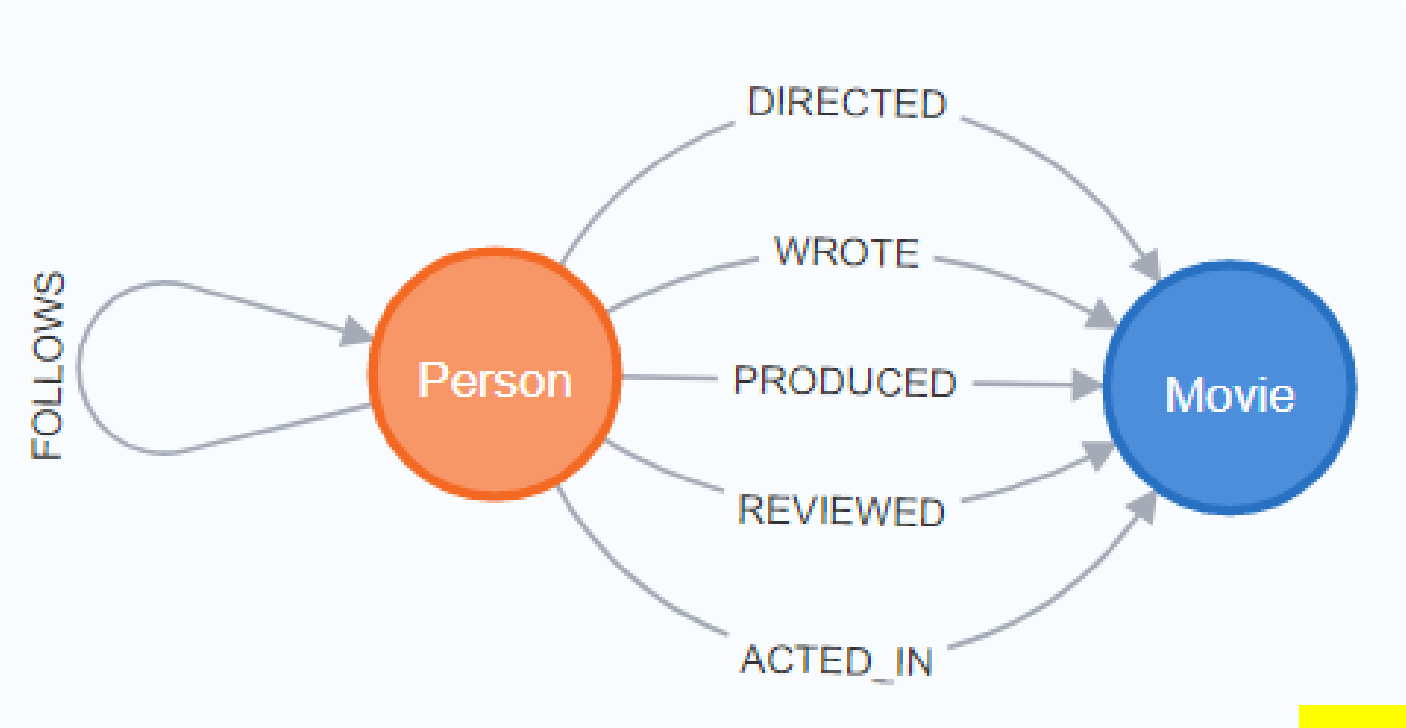
- Minden hivatkozásból él lesz
- Az éleket csak a csomópontok után tudjuk létrehozni, a belső azonosítók lekérdezésével

DEMO

Neo4j, Java alapú natív gráf adatbázis

- Hatékony bináris tárolás, külön a csomópontok, címkék, kapcsolatok (=élek), és attribútumok
- Egy csomópontnak több címkéje is lehet, NINCS node type
- Egy kapcsolatnak csak egy típusa lehet
- „opcionális” séma, kényszerek elhelyezhetőek
- Lekérdező nyelv: Cypher, mintaillesztésre alapul

Demo gráf adatbázis



DEMO